

Class 19 Windows Architecture & Registry For SOC L1

Windows Architecture

What is windows architecture?

Windows Architecture is the internal design of the Windows operating system. It defines how Windows manages its core components and how they interact.

It controls:

- **How processes run** – Creates, schedules, and manages running programs.
- **How memory is handled** – Allocates and protects RAM for each process.
- **How users authenticate** – Verifies user identities during login using credentials.
- **How security is enforced** – Applies permissions, access controls, and security policies to protect the system.

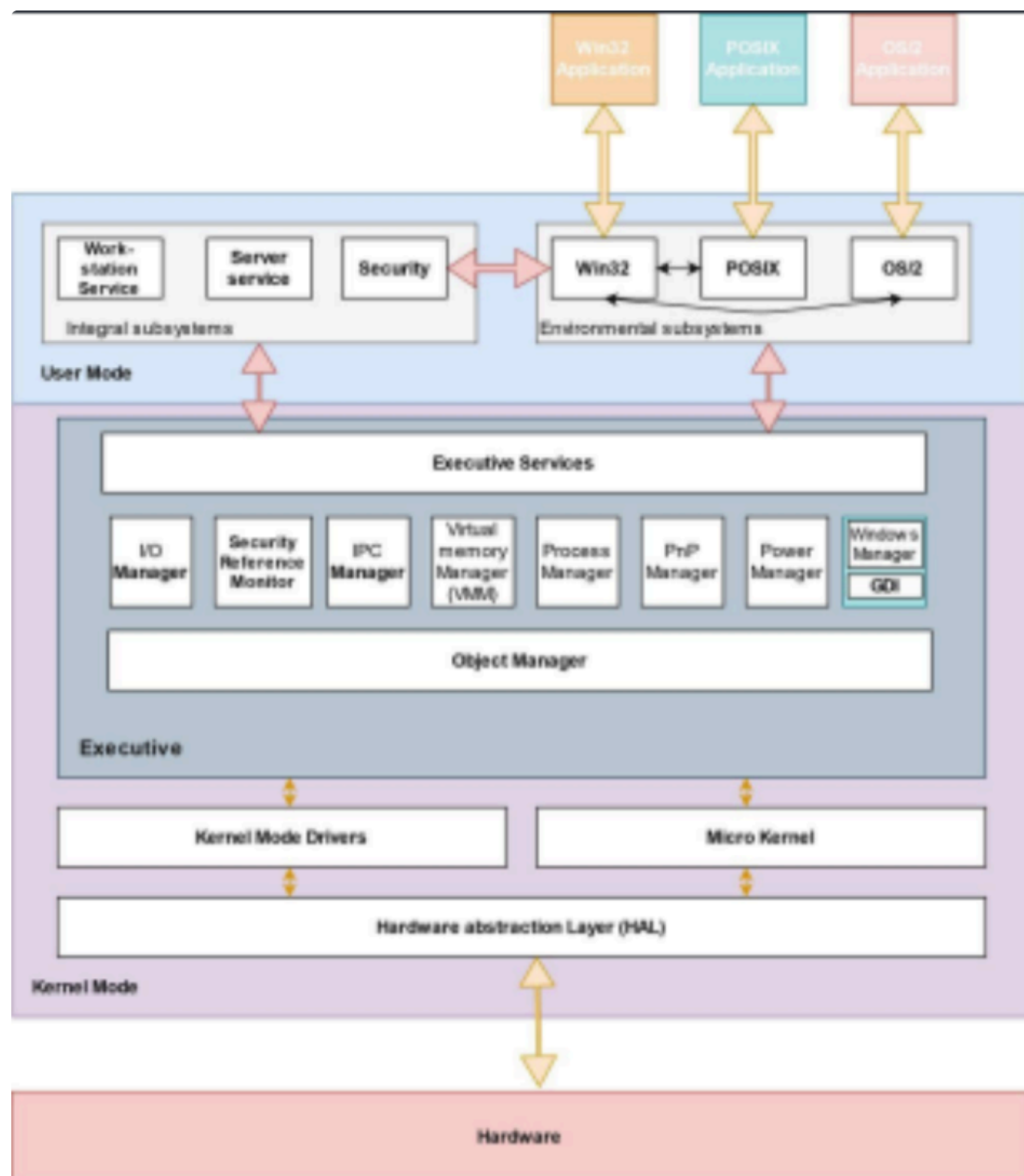
Why is it Important for SOC?

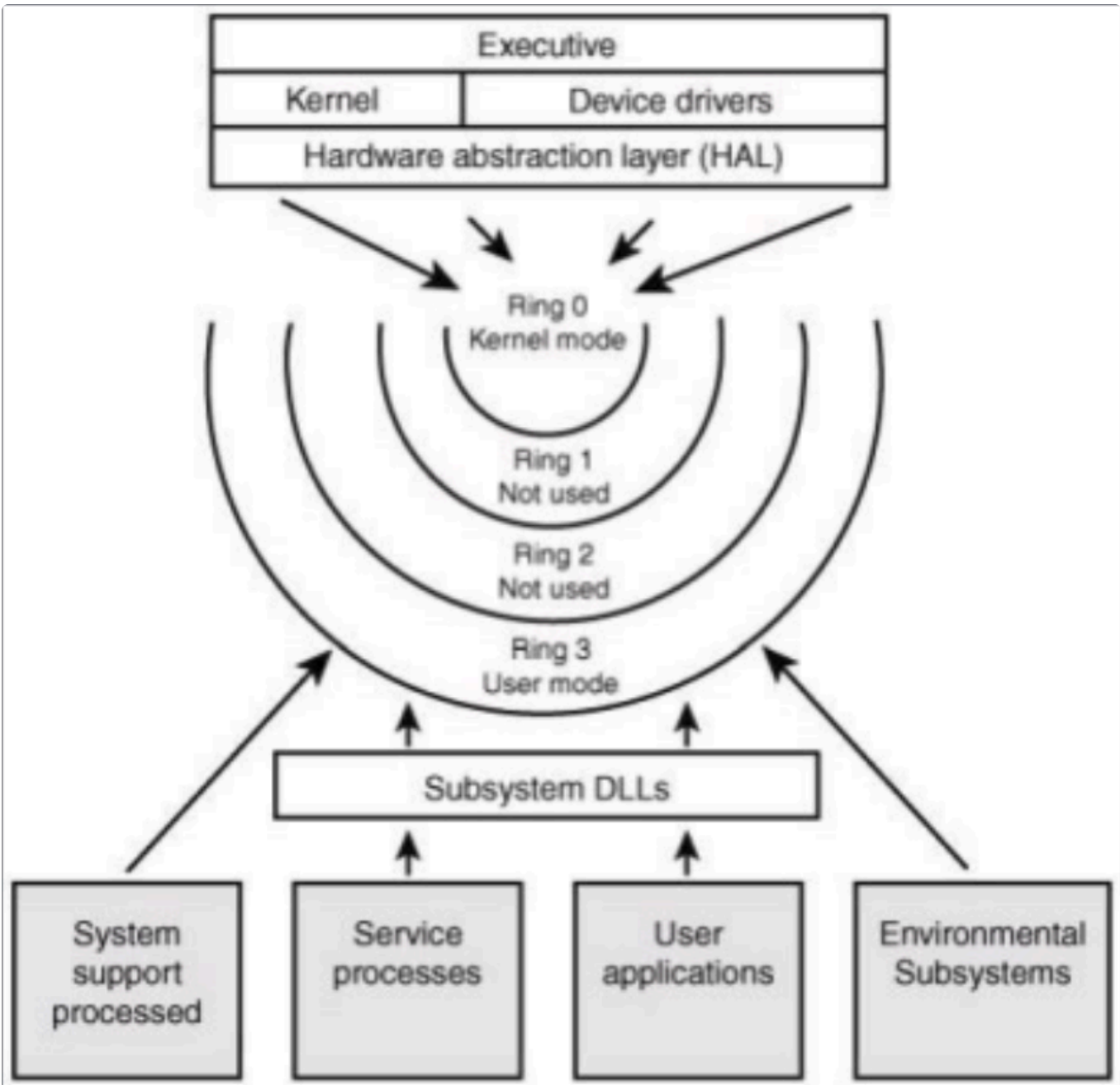
In **Security Operations Center (SOC)**, understanding Windows architecture helps analysts identify where attacks occur, such as:

- Malware running as a **process**.
- Credential theft during **authentication**.
- Malicious code injected into **memory**.
- Privilege escalation by bypassing **security mechanisms**.

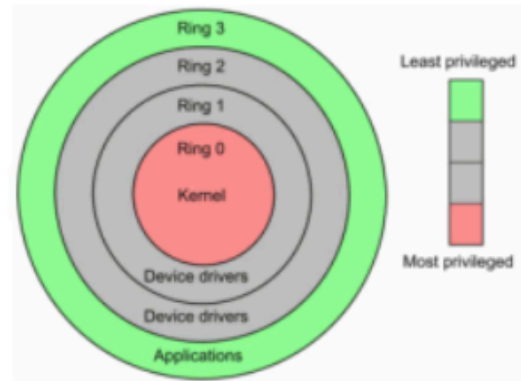
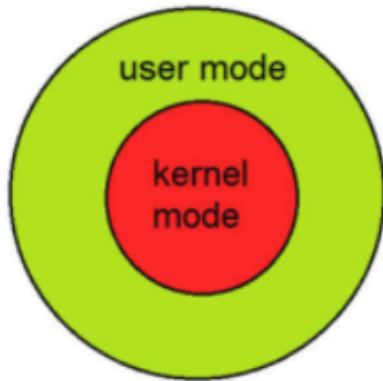
Summary:

Windows architecture explains **how Windows works internally**, and SOC analysts use this knowledge to detect, investigate, and respond to cyberattacks.





User Mode and Kernel Mode: Understanding Operating System Fundamentals



Two Core Modes

1. User Mode

What is User Mode?

User Mode is where normal applications and user programs run. It has **limited privileges**, so it cannot directly access hardware or modify the Windows kernel.

Runs:

- Applications (Chrome, Word, Excel, etc.)
- User processes
- Some Windows services

Restrictions:

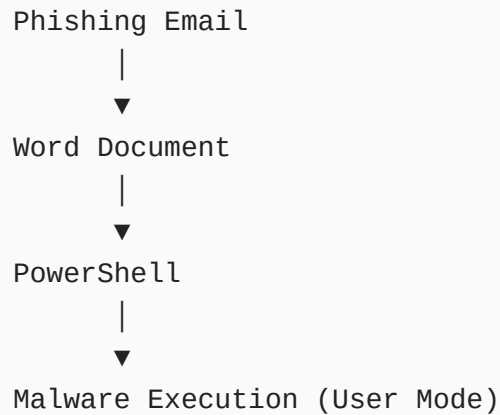
- Cannot directly access hardware.
- Cannot modify the Windows kernel.
- If an application crashes, it usually does **not** crash the entire operating system.

SOC Relevance:

- Most malware starts in **User Mode**.

- A common attack chain is:
 1. User opens a phishing email.
 2. A malicious Word document runs.
 3. It launches **PowerShell**.
 4. Malware executes in User Mode.

Flow:



2. Kernel Mode

What is Kernel Mode?

Kernel Mode is the most privileged part of Windows. It has **full access** to the CPU, memory, hardware, and all system resources.

Runs:

- Windows Kernel
- Device Drivers
- Memory Manager

Power:

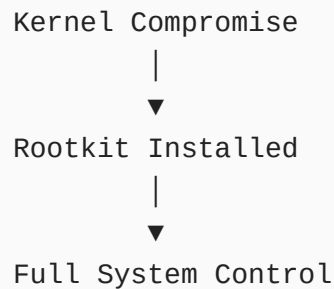
- Full system access.
- Can access hardware directly.
- Can modify any part of the operating system.

SOC Relevance:

- **Rootkits** often operate in Kernel Mode.

- If an attacker compromises the kernel, they can:
 - Hide malware and processes.
 - Disable security software.
 - Gain complete control of the system.

Flow:



Key Difference

User Mode	Kernel Mode
Runs applications	Runs Windows kernel and drivers
Limited privileges	Full system privileges
Cannot access hardware directly	Can access hardware directly
Most malware starts here	Rootkits and advanced malware operate here

Exam Tip

- User Mode = Limited access, where most applications and malware begin.
- Kernel Mode = Full system access; compromise here means complete control of the computer.

Windows Core Components

1. Windows Kernel

What is it?

The **Windows Kernel** is the core part of the operating system that communicates directly with the hardware and manages critical system functions.

Handles:

- **CPU Scheduling:** Decides which process or thread gets CPU time.
- **Interrupts:** Responds to hardware events (keyboard, mouse, disk, etc.).
- **Thread Execution:** Manages the execution of threads within processes.

SOC Insight:

- The kernel is **rarely attacked directly** because it is highly protected.
 - If an attacker compromises the kernel (e.g., using a **rootkit**), they can hide malware and gain stealthy, full-system control.
-

2. Executive Layer

The **Executive Layer** contains Windows components that manage processes, memory, security, and other operating system services.

A) Process Manager

What does it do?

- Creates, starts, and terminates processes.
- Manages process execution.

SOC Insight:

SOC analysts monitor for:

- Unusual process creation.
- Suspicious parent-child processes.
- Malware spawning processes like **PowerShell**, **cmd.exe**, or **rundll32.exe**.

Example:

```
WINWORD.EXE
  |
  ▼
powershell.exe
  |
  ▼
Malware
```

This is suspicious because Word normally should not launch PowerShell.

B) Memory Manager

What does it do?

- Allocates and manages RAM for processes.
- Protects one process's memory from another.

SOC Insight:

- **Fileless malware** executes directly in memory instead of saving malicious files to disk.
- Since no file is written, it is harder for traditional antivirus software to detect.

Example:

```
PowerShell
  |
  ▼
Loads malicious code into RAM
  |
  ▼
Runs without creating a file
```

1. I/O (Input/Output) Manager

What does it do?

The **I/O Manager** handles all **input and output operations** between applications and hardware.

It manages:

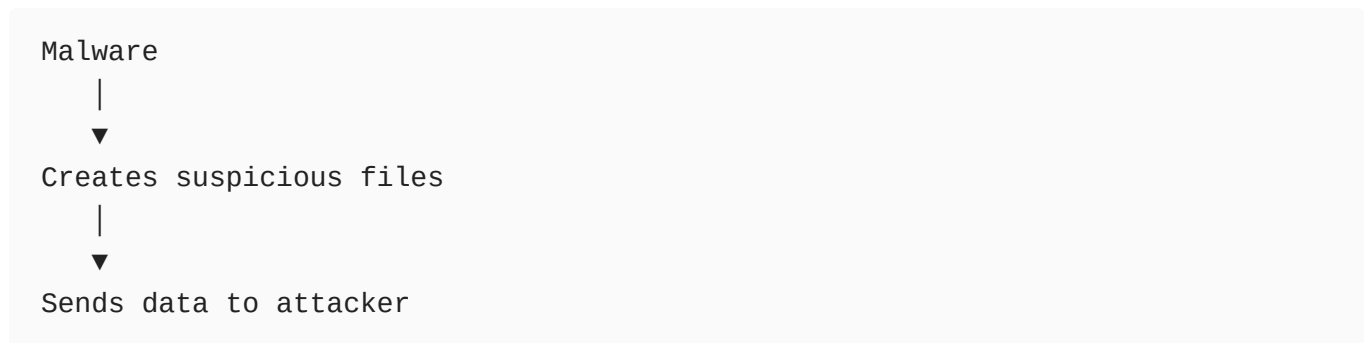
- **Disk operations** (read/write files)
- **Network operations** (sending/receiving data)
- Communication with device drivers

SOC Insight:

SOC analysts monitor the I/O Manager for:

- Unusual file creation, deletion, or modification.
- Suspicious network connections.
- Large or unexpected data transfers (possible data exfiltration).

Example:



2. Security Reference Monitor (SRM)

What is it?

The **Security Reference Monitor (SRM)** is the **core security engine** in Windows.

What does it check?

Whenever a user or process tries to access a resource (file, folder, registry, process, etc.), SRM asks:

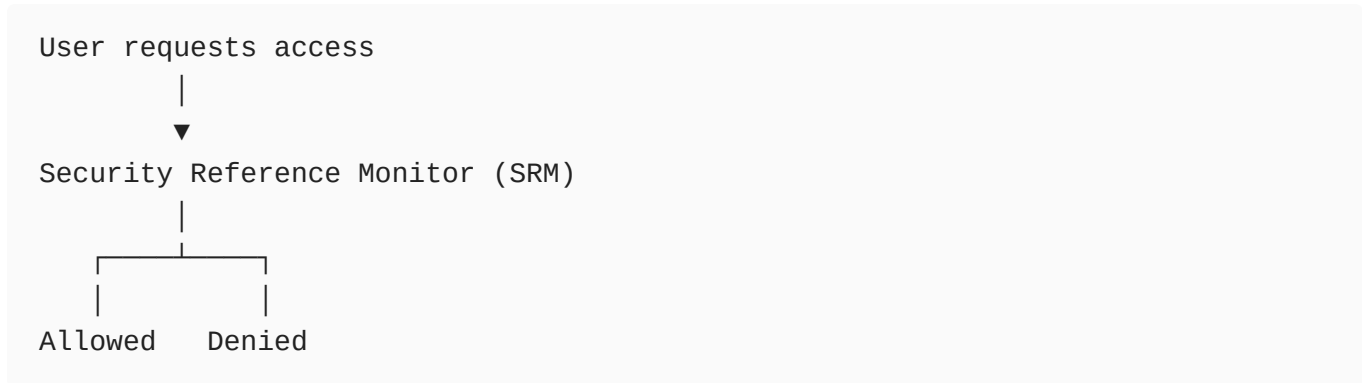
| "Does this user have permission to access this object?"

If permission exists, access is allowed. Otherwise, access is denied.

SOC Use:

- Detect **privilege escalation** (a user gaining higher privileges without authorization).
- Detect **unauthorized access** to sensitive files, folders, or system resources.

Example:



1. HAL (Hardware Abstraction Layer)

What is it?

The **Hardware Abstraction Layer (HAL)** acts as a bridge between the **Windows operating system** and the **computer hardware**.

It allows Windows to communicate with hardware without needing to know the hardware's specific details.

Flow:



SOC Relevance

- Advanced **rootkits** may target or abuse the HAL to hide malicious activity.

- Since HAL operates at a low level, compromising it can help attackers evade detection.

2. Device Drivers

What are they?

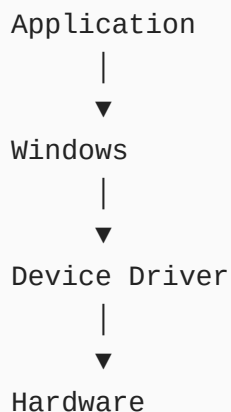
Device drivers are software components that allow Windows to communicate with hardware devices such as:

- Keyboard
- Mouse
- Printer
- Network card
- Graphics card

How they work

1. An application requests a hardware operation.
2. Windows sends the request to the appropriate driver.
3. The driver communicates with the hardware.
4. The hardware performs the requested action.

Flow:



Threat

- **Vulnerable drivers** can contain security flaws.
- Attackers can exploit these vulnerabilities to perform **privilege escalation**, allowing them to gain **administrator or SYSTEM-level**

privileges.

SOC Relevance

SOC analysts monitor for:

- Unsigned or suspicious drivers.
- Unexpected driver installations.
- Driver exploitation attempts leading to privilege escalation.

Windows Process Architecture

What is a Windows Process?

A **Windows process** is a running instance of a program. It acts as a container that holds all the resources required for the program to execute.

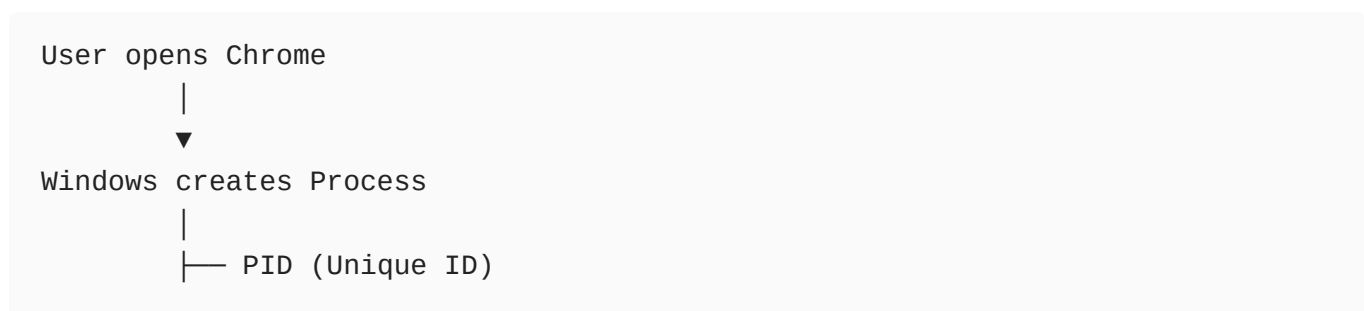
Each process has:

- **PID (Process Identifier)**: A unique number assigned by Windows to identify the process.
- **Memory**: Space allocated in RAM for the process to store code and data.
- **Threads**: One or more execution units that perform the actual work inside the process.

How it Works

1. The user starts a program (e.g., Chrome).
2. Windows creates a **process**.
3. A unique **PID** is assigned.
4. Memory is allocated.
5. One or more **threads** are created to execute the program.

Flow:



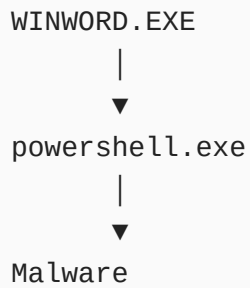
└─ Memory
└─ Threads

SOC Relevance

SOC analysts monitor processes to detect:

- Suspicious or unknown processes.
- Abnormal parent-child process relationships.
- Malware creating unexpected processes (e.g., `winword.exe` → `powershell.exe`).
- High CPU or memory usage caused by malicious processes.

Example:



1. `smss.exe` (Session Manager Subsystem)

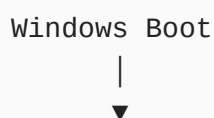
What does it do?

`smss.exe` is the **first user-mode process** started by the Windows kernel during boot.

How it works

1. Windows kernel finishes loading.
2. `smss.exe` starts.
3. It creates system sessions.
4. It launches important processes such as `wininit.exe` and `csrss.exe`.
5. After initialization, it waits for new user sessions.

Flow:



```
Windows Kernel
  |
  ▼
smss.exe
  |
  ▼
Starts wininit.exe & csrss.exe
```

SOC Relevance

- Normally only **one instance** runs.
- Multiple or suspicious `smss.exe` processes may indicate malware.

2. `wininit.exe` (Windows Initialization Process)

What does it do?

`wininit.exe` starts critical Windows system processes after `smss.exe`.

How it works

1. `smss.exe` starts `wininit.exe`.
2. `wininit.exe` launches:
 - `services.exe`
 - `lsass.exe`
 - `lsm.exe` (Local Session Manager)

Flow:

```
smss.exe
  |
  ▼
wininit.exe
  |
  ├───> services.exe
  ├───> lsass.exe
  └───> lsm.exe
```

SOC Relevance

- If `wininit.exe` is missing or replaced, Windows may not boot correctly.
 - Malware pretending to be `wininit.exe` is suspicious.
-

3. `services.exe` (Service Control Manager)

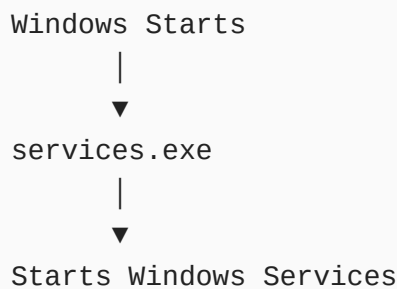
What does it do?

`services.exe` manages all Windows services.

How it works

1. Windows starts `services.exe`.
2. It reads the configured services.
3. Starts required services automatically.
4. Stops or restarts services when needed.

Flow:



SOC Relevance

Watch for:

- New unknown services.
 - Services created by malware.
 - Disabled security services.
-

4. `lsass.exe` (Local Security Authority Subsystem Service)

What does it do?

`lsass.exe` is responsible for Windows security.

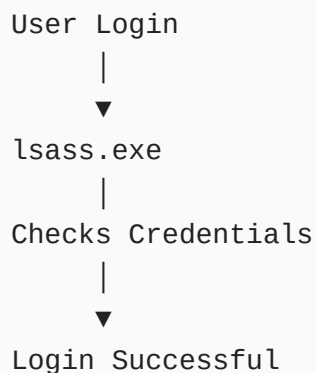
Handles

- User authentication.
- Password verification.
- Credential storage.
- Security policies.

How it works

1. User enters username and password.
2. `lsass.exe` verifies the credentials.
3. If valid, Windows creates a user session.
4. User gains access.

Flow:



SOC Relevance

Attackers use tools like **Mimikatz** to dump credentials from `lsass.exe` memory.

SOC analysts monitor for:

- Unauthorized access to `lsass.exe`.
 - Credential dumping attempts.
 - Unusual memory reads targeting `lsass.exe`.
-

5. `svchost.exe` (Service Host)

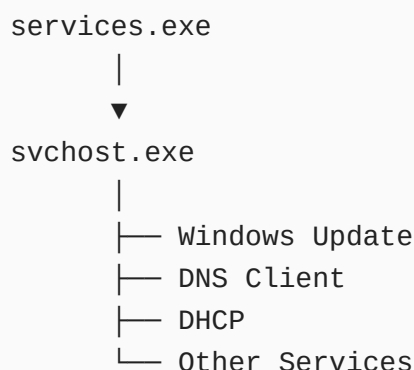
What does it do?

`svchost.exe` hosts one or more Windows services instead of each service running in its own process.

How it works

1. `services.exe` starts services.
2. Related services are grouped.
3. They run inside one or more `svchost.exe` processes.

Flow:



SOC Relevance

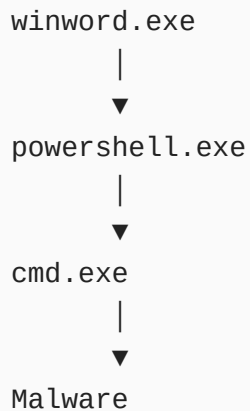
Malware often injects itself into `svchost.exe` because it is a trusted Windows process.

Watch for:

- `svchost.exe` with unusually high CPU or memory usage.
- Unexpected network connections.
- `svchost.exe` running from a location other than `C:\Windows\System32`.

SOC Detection Example

Attack Chain



How it works

1. The user opens a malicious Word document.
2. A macro inside the document runs automatically.
3. The macro launches `powershell.exe`.
4. PowerShell downloads or executes malicious code.
5. `cmd.exe` may be used to run additional commands.

Why is it suspicious?

Normally:

- `winword.exe` should edit documents.
- It should **not** start PowerShell or Command Prompt.

This unusual **parent-child process chain** is a common indicator of **malicious macro execution**, and SOC analysts frequently investigate it.

Windows Memory Architecture

Virtual Memory

What is Virtual Memory?

Virtual Memory is a memory management feature where **each process gets its own isolated memory space**.

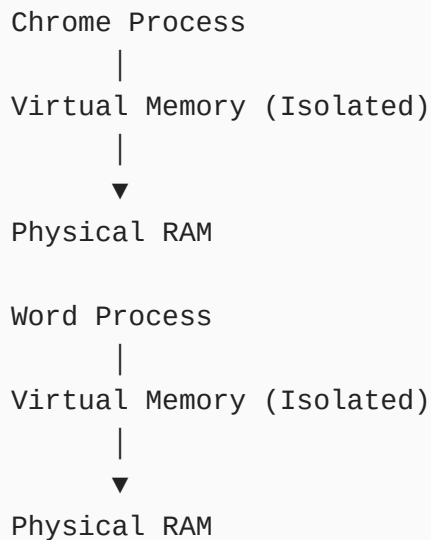
This means:

- One process **cannot directly access** another process's memory.
- Windows maps **virtual memory** to **physical RAM**.
- This improves **security** and **system stability**.

How it Works

1. A program (e.g., Chrome) starts.
2. Windows creates a separate virtual memory space for it.
3. Another program (e.g., Word) gets its own separate memory space.
4. The Memory Manager maps these virtual addresses to physical RAM.

Flow:



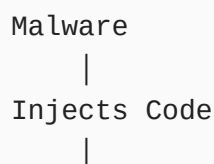
SOC Threats

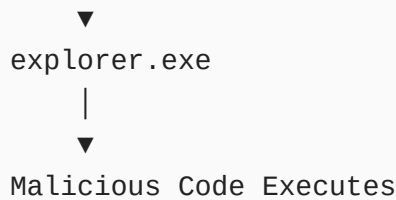
1. Process Injection

How it works

1. Malware starts on the system.
2. It injects its malicious code into a **legitimate process** (e.g., `explorer.exe`).
3. The legitimate process executes the malicious code.
4. The malware hides behind the trusted process.

Flow:





SOC Relevance

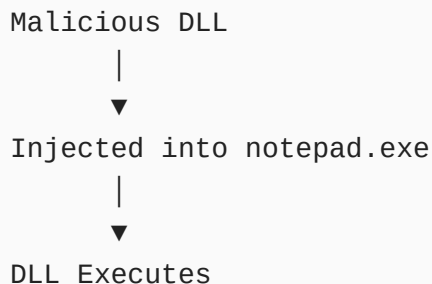
- Unexpected memory modifications.
 - Legitimate processes behaving abnormally.
-

2. DLL Injection

How it works

1. The attacker creates a malicious **DLL (Dynamic Link Library)**.
2. The DLL is injected into a legitimate process.
3. The process loads and executes the malicious DLL.
4. The attacker gains code execution under a trusted process.

Flow:



SOC Relevance

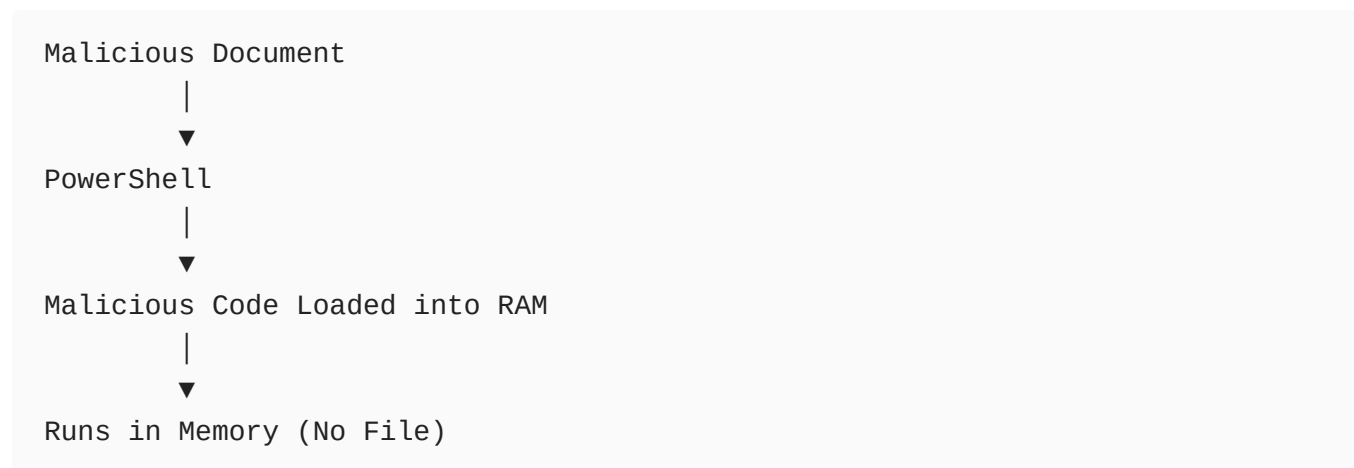
- Unsigned or suspicious DLLs.
 - DLLs loaded from unusual locations.
-

3. Fileless Malware

How it works

1. The user opens a malicious document or script.
2. PowerShell or another trusted tool downloads malicious code.
3. The code runs **directly in RAM** without creating a file on disk.
4. Traditional antivirus may not detect it because no malicious file exists.

Flow:



SOC Relevance

- Suspicious PowerShell activity.
- High memory usage.
- Malicious code executing only in RAM.

Windows Authentication Flow

Authentication Flow

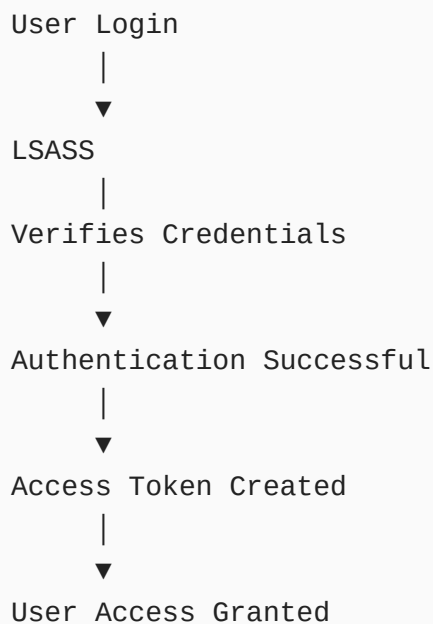
When a user logs into Windows, the operating system verifies the user's identity before granting access.

How it Works

1. **User Login** – The user enters a username and password.
2. **LSASS (lsass.exe)** – Receives the login request and verifies the credentials.
3. **Authentication** – Windows checks the credentials against the local database or Active Directory.
4. **Access Token Created** – If authentication is successful, Windows creates an **Access Token**.

5. **Access Granted** – Every process started by the user uses this access token to determine what resources the user can access.

Flow:



Access Token

What is an Access Token?

An **Access Token** is a security object created after a successful login. It represents the user's identity and permissions.

It Contains:

- **User Identity** (User SID)
- **User Privileges** (permissions and access rights)
- Group memberships (e.g., Administrators)

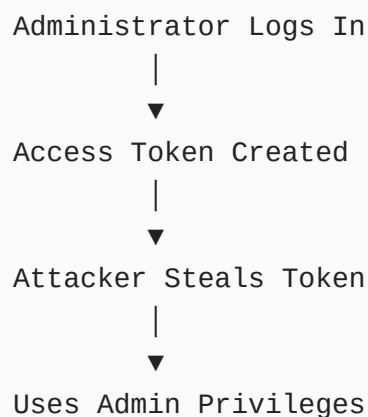
Whenever the user opens a program, Windows checks the **Access Token** to decide whether the user is allowed to access a file, folder, registry key, or other system resource.

SOC Use – Token Theft

How it Works

1. A legitimate user logs in.
2. Windows creates an access token.
3. Malware steals or duplicates the access token.
4. The attacker uses the stolen token to act as the legitimate user.
5. If the token belongs to an administrator, the attacker gains **privilege escalation** without needing the password.

Flow:



SOC Indicators

- Unexpected privilege escalation.
- Token impersonation events.
- Unusual logon sessions.
- Processes running with elevated privileges unexpectedly.

Windows Registry

What is the Windows Registry?

The **Windows Registry** is a **central database** that stores configuration settings and options for the Windows operating system, hardware, applications, and user accounts.

It stores:

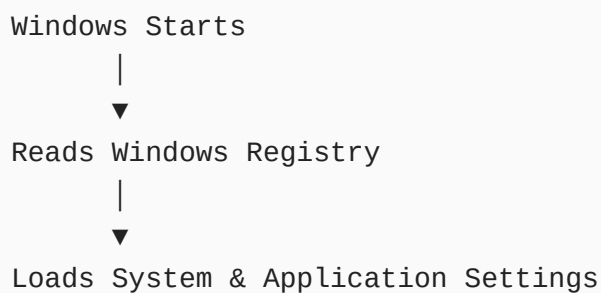
- **System configuration**
- User settings
- Installed software information

- Hardware configuration
- Security settings

How it Works

1. Windows starts.
2. It reads configuration settings from the Registry.
3. Applications also read and write settings to the Registry while running.
4. Windows uses these settings to control system behavior.

Flow:



SOC Relevance

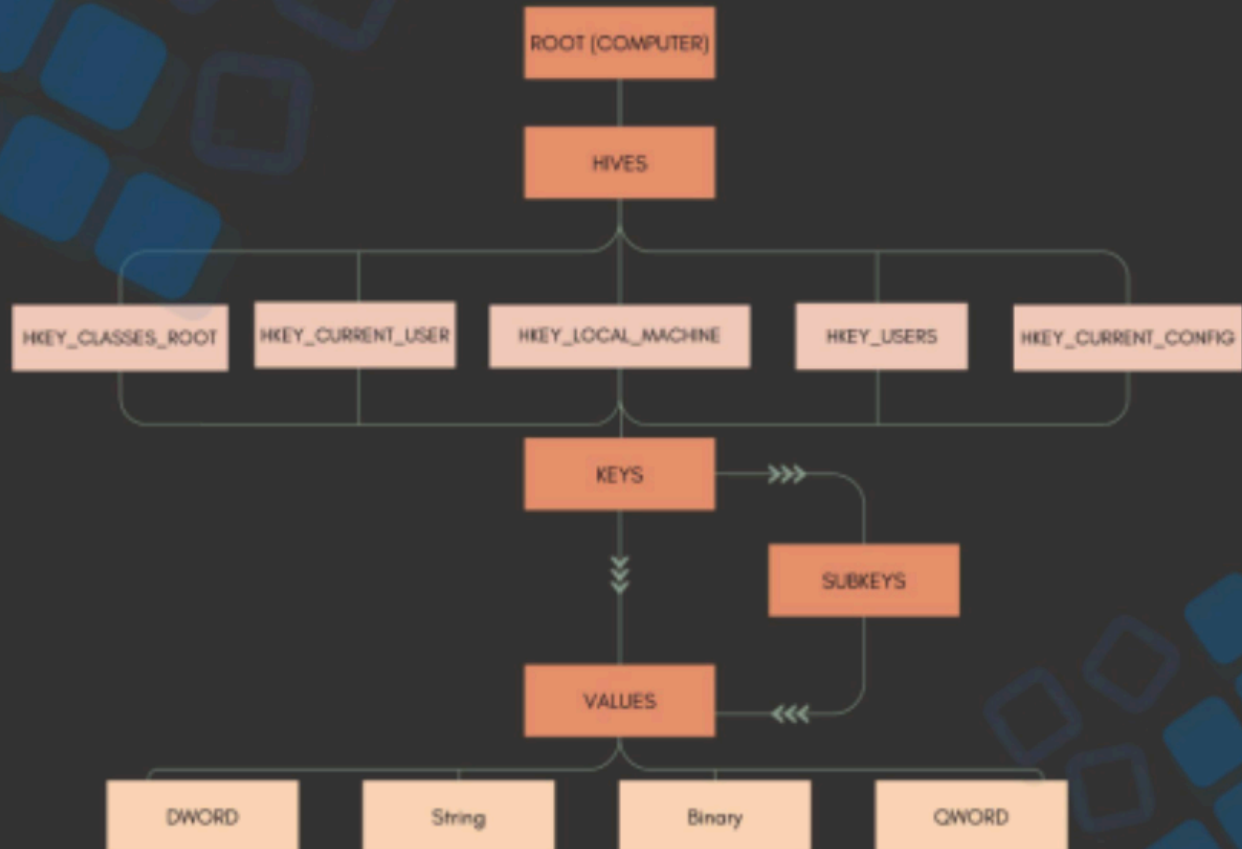
Attackers often modify the Registry to:

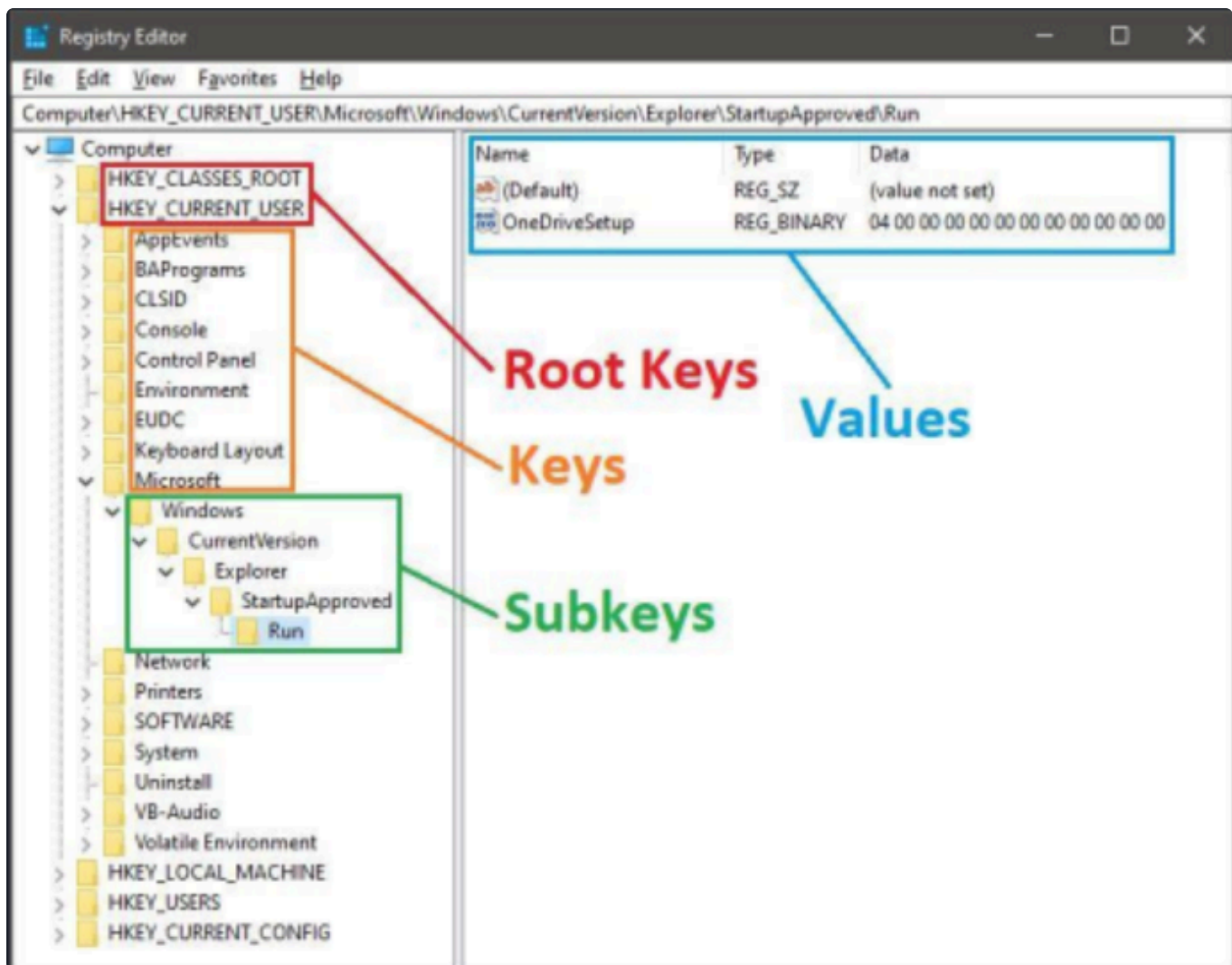
- **Maintain persistence** (run malware automatically at startup).
- Change security settings.
- Disable antivirus or security tools.
- Hide malicious activity.

Example

A malware program adds itself to the Registry's **Run** key so it launches automatically every time the computer starts.

Windows Registry Basic Hierarchical Chart





Registry Structure(Root Keys (Hive))

HIVE	MEANING
HKLM	System-Wide
HKCU	Current User
HKCR	File associations
HKU	All users
HKCC	Hardware config

HIVE -> Key -> Subkey -> Value

Windows Registry - How It Works (Short Explanation)

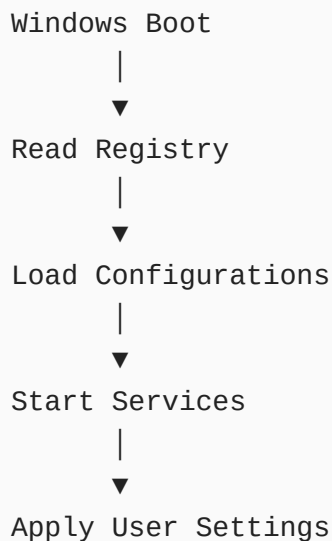
How the Registry Works

The **Windows Registry** stores configuration settings used by Windows and applications.

During System Boot

1. **Windows reads the Registry.**
2. **Loads system configurations** (startup settings, drivers, hardware settings).
3. **Starts Windows services** based on Registry entries.
4. **Applies user settings** such as desktop, network, and personalization.

Flow:

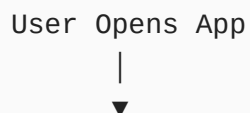


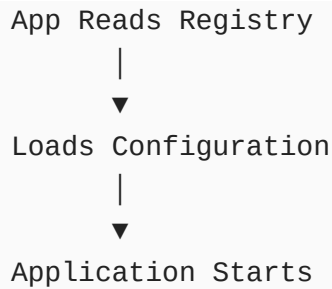
When an Application Runs

How it Works

1. The user opens an application (e.g., Chrome or Microsoft Word).
2. The application reads its settings from the Registry.
3. It loads the required configuration and starts normally.

Flow:





Types of Registry Values

1. REG_SZ (String)

- Stores **text or string values**.
- **Example:** Username, file path, or application name.

2. REG_DWORD (32-bit Number)

- Stores **numeric values**.
- Commonly used for **enable/disable settings** (e.g., 0 = Disabled, 1 = Enabled).

3. REG_BINARY

- Stores **binary (hexadecimal) data**.
- Used for hardware settings, device configurations, and other low-level data.

SOC Relevance

SOC analysts monitor Registry changes because malware often:

- Adds **startup (Run) keys** for persistence.
- Modifies security settings.
- Disables antivirus or Windows Defender.
- Changes application configurations to maintain access.

Windows Registry

Attackers use these Registry locations to **maintain persistence**, meaning their malware continues to run even after the computer is restarted or the user logs in again.

1. Run Key

Location:

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Run
```

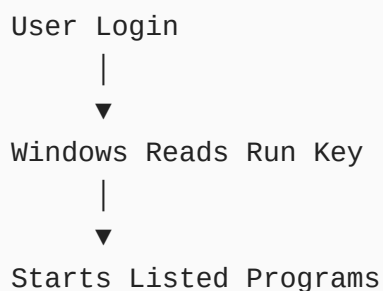
Purpose

The **Run** key stores programs that Windows automatically starts **every time the user logs in**.

How it Works

1. User logs into Windows.
2. Windows checks the **Run** key.
3. Every program listed there is launched automatically.

Flow:



Attack

The attacker adds:

```
malware.exe
```

to the Run key.

Now, **every login automatically starts the malware**, giving the attacker persistence.

SOC Relevance

- Unexpected entries in the Run key.
 - Unknown executables starting at login.
-

2. RunOnce Key

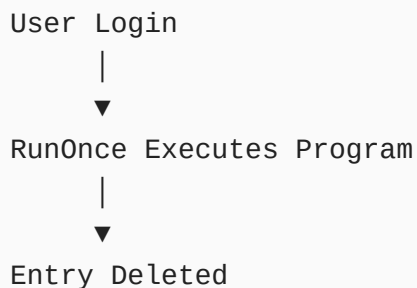
Purpose

The **RunOnce** key stores programs that should run **only one time**, usually after the next login or reboot.

How it Works

1. User logs in.
2. Windows executes the program listed in **RunOnce**.
3. Windows automatically removes the entry after it runs.

Flow:



Attack

Malware may use **RunOnce** to:

- Execute a payload once.
- Download additional malware.
- Change system settings.

SOC Relevance

- Unexpected RunOnce entries.
 - Suspicious scripts or installers executing after reboot.
-

3. Services Registry

Location:

```
HKLM\SYSTEM\CurrentControlSet\Services
```

Purpose

This Registry key stores information about **Windows services**.

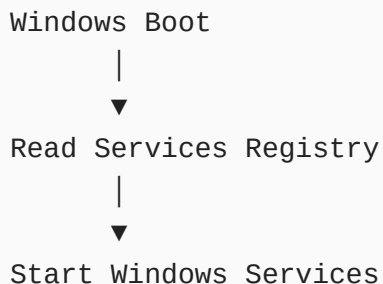
It defines:

- Which services exist.
- Whether they start automatically.
- The executable file used by each service.

How it Works

1. Windows starts.
2. It reads the **Services** Registry key.
3. It starts services configured for automatic startup.

Flow:



Attack

An attacker creates a **malicious Windows service**.

Because services can start automatically with Windows, the malware runs **before the user even logs in**, making it harder to notice and remove.

SOC Relevance

- New or unknown services.

- Services pointing to suspicious executables.
- Services with unusual startup types.

Registry & Attacks

1. WMI Persistence

Purpose: Advanced persistence technique.

How it works:

- Attackers create a **WMI (Windows Management Instrumentation)** event that automatically runs malware when a specific event occurs (e.g., user login or system startup).
- Unlike the **Run** key, it is harder to detect because it doesn't rely on common startup locations.

SOC Use: Monitor for suspicious WMI event subscriptions.

2. Registry Hijacking

Purpose: Execute malicious code instead of legitimate programs.

How it works:

- The attacker modifies Registry entries that point to a legitimate executable or DLL.
- When Windows or an application starts, it loads the **malicious file** instead of the legitimate one.

SOC Use: Monitor Registry changes to important application paths.

SOC Detection – What to Monitor

Suspicious Entries

- Example: `powershell.exe` in the **Run** key.
- Indicates PowerShell may be configured to run automatically at login.

Unknown Executables

- Example: `random.exe` set to auto-start.
- Unknown programs starting automatically may indicate malware.

Unusual Paths

- Example:

```
C:\Users\<User>\AppData\Temp\
```

- Malware often runs from **AppData** or **Temp** folders because they are writable by users and less likely to be noticed.

Summary

- **WMI Persistence:** Malware runs automatically using WMI events.
- **Registry Hijacking:** Registry is modified to launch malicious files instead of legitimate ones.
- **SOC Monitoring:** Look for suspicious startup entries, unknown executables, and programs launching from unusual locations like **AppData** or **Temp**.

Registry Forensics

Registry Forensics is the process of analyzing the Windows Registry to investigate user activity and detect malicious actions.

What can you extract?

- **Installed Software:** Find what programs are installed on the system.
- **User Activity:** Determine which users logged in, opened files, or changed settings.
- **Execution History:** Identify which applications or commands have been executed.

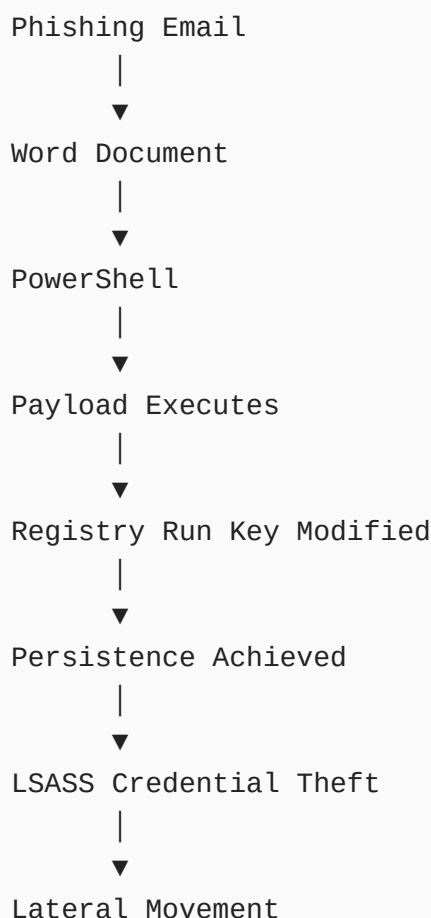
SOC Use: Helps analysts investigate malware infections and reconstruct attacker activity.

Windows Architecture + Registry Attack Flow

How the Attack Works

1. **Phishing** - The user opens a malicious email attachment.
2. **Word** - The malicious Word document contains a macro.
3. **PowerShell** - The macro launches PowerShell.
4. **Payload Executes** - PowerShell downloads or runs the malware.
5. **Registry Run Key Modified** - Malware adds itself to the **Run** key for persistence.
6. **Persistence Achieved** - Malware starts automatically every time the user logs in.
7. **LSASS Attacked** - The attacker steals user credentials (e.g., using Mimikatz).
8. **Lateral Movement** - The attacker uses the stolen credentials to access other systems on the network.

Flow:



Registry for SOC – Short Explanation

Event ID: 4657

Event ID 4657 indicates that a Windows Registry value was modified.

Example:

```
Event ID: 4657
Registry Key: HKCU\...\Run
Value: malware.exe
```

What it Means

- A Registry key has been changed.
- If `malware.exe` is added to the **Run** key, it will execute automatically whenever the user logs in.
- **Result: Persistence detected** (the malware survives reboots and logins).

SOC Action

- Verify who modified the Registry.
 - Check whether the executable is legitimate.
 - Investigate related processes and user activity.
-

Final SOC Mindset

When analyzing a Windows system, ask these five questions:

1. Which process ran?

- Identify the suspicious process (e.g., `powershell.exe`, `cmd.exe`, `winword.exe`).

2. What did it modify?

- Check if it modified files, Registry keys, services, or scheduled tasks.

3. Any Registry persistence?

- Look for startup entries such as:
 - `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`
 - `RunOnce`
 - Services Registry

4. Any credential access?

- Check whether `lsass.exe` was accessed or if credential-dumping tools (e.g., Mimikatz) were used.

5. Any lateral movement?

- Look for signs the attacker moved to other systems using:
 - RDP
 - SMB
 - PsExec
 - Stolen credentials

Summary

A SOC analyst should think in this order:

Suspicious Process



What Changed?



Persistence?



Credential Theft?



Lateral Movement?

This sequence helps analysts reconstruct the attack chain, determine the attacker's objectives, and identify the full scope of the compromise.